# Yanbu Industrial College

الهيئة الملكية للجبيل وينبع
Royal Commission for Jubail & Yanbu

Computer Science And Engineering Department

Academic Year 1447- 1448 / 2025- 2026

First Semester

Project 1

Programming Languages

CS360

| Due Date: | 11/24/2025 | 2Choose an item. |
|---|---|---|

Student Name: Sarah [redacted] & Amjad [redacted] ID Number: [redacted]

Teacher's Name: Ms.Omamah Section: 2

Please read the following instructions carefully:

- Write your name, I.D. number and section number in the space provided above.
- Read each questio[redacted]efore ans[redacted]
- Answer all questions.

[redacted]

For Instructor Use Only

| CLO. | Question No. | Mark Assigned | Mark Attained |
|---|---|---|---|
| 3.01<br>3.02 | Q1 | 10 | |
| 2.01 | Q2 | 10 | |
| | | | |
| | | | |
| | | | |
| | | Total: | |

Marked by: Ms.Omamah Signature:

[Python]

# Research

Python is a high-level, general-purpose programming language created by Guido van Rossum in 1989 and officially released in 1991. The main goal behind Python's design was to create a language that is simple, clear, and easy to read. Over the years, Python has become one of the most popular languages in both education and industry because of its clean syntax, strong community, and wide range of applications.

Python was developed at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. Guido van Rossum wanted a language that was easier than C and simpler than Perl, so he designed Python using English-like keywords and a focus on readability. Python 2 was released in 2000, followed by Python 3 in 2008, which improved the language and made it more consistent. Today, Python continues to grow rapidly and is used worldwide in scientific research, data science, artificial intelligence, and education.

Readability:

Python is widely known as one of the most readable programming languages. Its syntax is clear and resembles natural English, which makes the code easy to understand even for beginners. Python uses indentation instead of braces to define blocks, which forces clean and consistent formatting. Variable declarations are simple, and the language avoids unnecessary symbols, making programs visually clean.

Summary:

Python has excellent readability, which is one of the reasons it is often recommended as a first programming language.

Writability:

Writability refers to how quickly and easily a programmer can write code. Python offers very high writability because: It does not require variable type declarations It uses short and simple statements Many tasks can be done in fewer lines compared to other languages Built-in libraries make writing code easier and faster.

Summary:

Python allows developers to write programs quickly with minimal syntax, making it highly productive.

Reliability:

Python is considered reliable, especially for education, data science, automation, and web development. The language provides clear error messages, has a large set of stable standard libraries, and is supported by a huge community. However, because Python is an interpreted language, it is not always the fastest, and performance-critical systems may prefer languages like C++ or Java.

Summary:

Python is reliable for most applications, especially where development speed and clarity are more important than raw performance.

Cost:

Python is considered a very low-cost language. Because it is open-source and completely free, organizations do not need to pay for licenses or commercial tools. Most Python IDEs, such as Thonny, IDLE, and VS Code, are also free to use. In addition, Python has a low learning cost because the language is simple, easy to understand, and reduces the training required for new developers. In addition, Python has a low learning cost because the language is simple, easy to understand, and reduces the training required for new developers.

Portability:

Python programs are portable and can run on multiple operating systems including Windows, macOS, and Linux without requiring code changes. This flexibility makes Python a strong choice for developers who need to build applications that work across different platforms.

Popularity:

Python has one of the largest developer communities worldwide. This community maintains thousands of libraries, contributes to documentation, and provides support through forums, tutorials, and online courses. Strong community support increases reliability, reduces development problems, and helps beginners learn faster.

Python’s Role in Artificial Intelligence Research:

Python has become the dominant language in AI research due to its simplicity, flexibility, and powerful ecosystem of scientific libraries. Research papers published in the fields of

machine learning, neural networks, and natural language processing often rely on Python frameworks such as TensorFlow, PyTorch, Scikit-Learn, and Keras. These libraries allow researchers to develop complex models with fewer lines of code and make experimentation faster and more accessible. Scientific studies also highlight that Python's syntax helps researchers focus on the mathematical concepts behind AI rather than dealing with low-level programming details.

Programming Paradigms:

Python is a multi-paradigm programming language, meaning it supports different programming styles:

• Procedural Programming

Writing code step-by-step, which is perfect for beginners and simple tasks.

• Object-Oriented Programming

Supports classes, objects, inheritance, and encapsulation.

• Functional Programming

Includes features like lambda functions, map(), filter(), and higher-order functions.

This flexibility makes Python more adaptable than languages like Java, which is primarily object-oriented.

Strengths and Weaknesses

Strengths:

- Very high readability
- Easy to write and understand
- Large standard library and community support
- Cross-platform and open-source
- Strong in data science, AI, and automation
- Perfect for beginners

Weaknesses:

- Slower than languages like C++ and Java
- Not ideal for low-level or real-time systems
- Some libraries may require external dependencies

- Large programs can be slower due to dynamic typing

Applications and Organizations Using Python:

Python is used in a wide range of fields:

Applications

- Machine Learning & AI
- Data Analysis & Visualization
- Web Development
- Automation / Scripting
- Cybersecurity tools
- Scientific Computing
- Game Prototyping
- Education

Organizations

Many major companies rely heavily on Python, including:

- Google
- NASA
- Netflix
- Instagram
- Spotify
- Meta (Facebook)
- Dropbox
- Reddit

These organizations use Python because it is efficient, flexible, and easy to maintain.

Conclusion

Python combines simplicity, readability, and strong functionality, making it one of the most widely used programming languages today. Its high readability and writability allow beginners to learn quickly, while its reliability makes it suitable for real applications in major companies. For this project, Python was an excellent choice because it allowed us to build a simple and clear program while strengthening our understanding of programming fundamentals.

# Project

Our project is a simple Rock–Paper–Scissors game made using Python.
The idea of the project is to practice the basic concepts of Python, such as printing, taking input from the user, using conditions, loops, and simple logic.
The game works in small rounds. Every time the player chooses Rock, Paper, or Scissors, the computer chooses randomly. Then the program compares both choices and shows who wins the round. The special part of our project is that the game plays three rounds at a time, and after each set of three rounds, it shows the total score for the player and the computer. The user can choose to continue playing with the same total score, or reset the score and start again, or exit the game. The idea is simple, clear, and beginner-friendly. It helped us understand how Python works and how we can use loops, conditions, and variables to build a small interactive program.

```
import random
print("Welcome to Rock, Paper, Scissors!")
total_user_wins = 0
total_comp_wins = 0
while True:
    rounds = 0
    print("\nStarting 3 rounds...")
    while rounds < 3:
        print("\nChoose your move:")
        print("1) Rock")
        print("2) Paper")
        print("3) Scissors")
        print("4) Exit")
        user = input("Enter your choice (1-4): ")
        if user == "4":
            print("Goodbye!")
```

```
        exit()
    if user == "1":
        user1 = "rock"
    elif user == "2":
        user1 = "paper"
    elif user == "3":
        user1 = "scissors"
    else:
        print("Invalid choice, try again.")
        continue
    comp = random.randint(1, 3)
    if comp == 1:
        comp1 = "rock"
    elif comp == 2:
        comp1 = "paper"
    else:
        comp1 = "scissors"
    print("\nYou chose: %s" % user1)
    print("Computer chose: %s" % comp1)
    if user1 == comp1:
        print("Result: It's a tie!")
    elif user1 == "rock" and comp1 == "scissors":
        print("Result: You win!")
        total_user_wins += 1
    elif user1 == "paper" and comp1 == "rock":
        print("Result: You win!")
        total_user_wins += 1
    elif user1 == "scissors" and comp1 == "paper":
        print("Result: You win!")
        total_user_wins += 1
    else:
```

```
        print("Result: Computer wins!")
        total_comp_wins += 1
    rounds += 1
print("\n==== TOTAL RESULT ====")
print("Your total wins: %d" % total_user_wins)
print("Computer total wins: %d" % total_comp_wins)

print("\nWhat do you want to do?")
print("1) Continue and keep the total score")
print("2) Delete total score and start from zero")
print("3) Exit game")
choice = input("Enter 1, 2, or 3: ")
if choice == "2":
    total_user_wins = 0
    total_comp_wins = 0
    print("Score deleted! Starting from zero.")
elif choice == "3":
    print("Goodbye!")
    break
```

project.py

```python
import random
print("Welcome to Rock, Paper, Scissors!")

total_user_wins = 0
total_comp_wins = 0

while True:
    rounds = 0

    print("\nStarting 3 rounds...")

    while rounds < 3:
        print("\nChoose your move:")
        print("1) Rock")
        print("2) Paper")
        print("3) Scissors")
        print("4) Exit")

        user = input("Enter your choice (1-4): ")

        if user == "4":
            print("Goodbye!")
            exit()

        if user == "1":
            user1 = "rock"
        elif user == "2":
            user1 = "paper"
        elif user == "3":
            user1 = "scissors"
        else:
            print("Invalid choice, try again.")
            continue
```

project.py

```
34
35          comp = random.randint(1, 3)
36          if comp == 1:
37              comp1 = "rock"
38          elif comp == 2:
39              comp1 = "paper"
40          else:
41              comp1 = "scissors"
42
43          print("\nYou chose: %s" % user1)
44          print("Computer chose: %s" % comp1)
45
46          if user1 == comp1:
47              print("Result: It's a tie!")
48          elif user1 == "rock" and comp1 == "scissors":
49              print("Result: You win!")
50              total_user_wins += 1
51          elif user1 == "paper" and comp1 == "rock":
52              print("Result: You win!")
53              total_user_wins += 1
54          elif user1 == "scissors" and comp1 == "paper":
55              print("Result: You win!")
56              total_user_wins += 1
57          else:
58              print("Result: Computer wins!")
59              total_comp_wins += 1
60
61          rounds += 1
62
63      print("\n==== TOTAL RESULT ====")
64      print("Your total wins: %d" % total_user_wins)
65      print("Computer total wins: %d" % total_comp_wins)
66
```

```
66
67      print("\nWhat do you want to do?")
68      print("1) Continue and keep the total score")
69      print("2) Delete total score and start from zero")
70      print("3) Exit game")
71
72      choice = input("Enter 1, 2, or 3: ")
73
74      if choice == "2":
75          total_user_wins = 0
76          total_comp_wins = 0
77          print("Score deleted! Starting from zero.")
78
79      elif choice == "3":
80          print("Goodbye!")
81          break
```

Output:

```
Welcome to Rock, Paper, Scissors!

Starting 3 rounds...

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 1

You chose: rock
Computer chose: rock
Result: It's a tie!

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 3

You chose: scissors
Computer chose: paper
Result: You win!

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 3

You chose: scissors
Computer chose: scissors
Result: It's a tie!
```

```
==== TOTAL RESULT ====
Your total wins: 1
Computer total wins: 0

What do you want to do?
1) Continue and keep the total score
2) Delete total score and start from zero
3) Exit game
Enter 1, 2, or 3: 1

Starting 3 rounds...

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 3

You chose: scissors
Computer chose: scissors
Result: It's a tie!

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 2

You chose: paper
Computer chose: rock
Result: You win!

Choose your move:
1) Rock
2) Paper
3) Scissors
```

```
You chose: rock
Computer chose: scissors
Result: You win!

==== TOTAL RESULT ====
Your total wins: 3
Computer total wins: 0

What do you want to do?
1) Continue and keep the total score
2) Delete total score and start from zero
3) Exit game
Enter 1, 2, or 3: 2
Score deleted! Starting from zero.

Starting 3 rounds...

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 2

You chose: paper
Computer chose: rock
Result: You win!

Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 3

You chose: scissors
Computer chose: rock
Result: Computer wins!
```

```
Choose your move:
1) Rock
2) Paper
3) Scissors
4) Exit
Enter your choice (1-4): 1

You chose: rock
Computer chose: rock
Result: It's a tie!

==== TOTAL RESULT ====
Your total wins: 1
Computer total wins: 1

What do you want to do?
1) Continue and keep the total score
2) Delete total score and start from zero
3) Exit game
Enter 1, 2, or 3: 3
Goodbye!
>>>
```

# Reference

https://inference-review.com/article/the-origins-of-python

https://inference-review.com/article/the-origins-of-python

https://inference-review.com/article/the-origins-of-python

Rubrics:

2.01 Design and develop small scale programming language.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Excellent | | Good | | Accepted | | Developing | | Below | |

3.03 Research alternative design approach

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Excellent | | Good | | Accepted | | Developing | | Below | |

Descriptions:

1- Below ~ attributes for evaluation not exist or exist but poorly stated or prepared (format not visible, obvious typo errors, obvious grammatical errors. Group did not turn up in exhibition/presentation/evident of unresolved conflict.
2- Developing ~ attributes visible but without details. Description may be vague and difficult to understand. Visible minor error. Last minute submission. Group member/s absence/s during exhibition/presentation.
3- Accepted ~ attributes visible with simple description and understandable. Exist and follow proper formatting with acceptable errors. Observe delays in submission. All group members are present and at least one of each member demonstrate min capability in delivering the content of the presentation.
4- Good ~ attributes has complete descriptions with supporting details and materials, in a proper format and fulfill report documentation. Contents are error free. Submission is on time. All members are present during exhibition/presentation and able to deliver and discuss their unit.
5- Excellent ~ demonstrate product uniqueness, item produced reaches to an exceed expectation. All members are present during exhibition/presentation and able to deliver/discuss confidently and correctly on their unit assigned.